



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
ASIGNATURA: LENGUAJES Y COMPILADORES

Tema 4:

ANALISIS LEXICO

DOCENTE:

Félix Márquez

ALUMNOS:

Daniel Villalba Santamaría **V- 27.506.542**

Isabella Rodríguez **V - 31.521.612**

Luis Millán **V - 30.040.213**

Andrés Gómez **V - 31.085.717**

Ciudad Guayana, Julio del 2026.

ÍNDICE

INTRODUCCIÓN	3
ACTIVIDAD I - AUTÓMATAS DE PILA.....	4
• Ejemplos de Aplicación y Funcionamiento Visual	5
ACTIVIDAD II - LEXER PARA ARCHIVOS DOCKERFILE MEDIANTE REGEX.....	8
• Diccionario de Tokens y Expresiones Regulares.....	9
• Implementación en Python y Construcción del Código	10
• Ejemplos de Ejecución y Manejo de Errores	19
ACTIVIDAD III - DESCRIPCIÓN FORMAL DEL LENGUAJE DISEÑADO (LRust).....	22
• Manual de Usuario del Metacompilador Flex	24
• Metodología y Proceso de Creación del Analizador Léxico	25
• Contenido del Archivo README.md (Instalación e Implementación)	27
• Selección del Lenguaje de Dominio Específico (DSL)	30
ACTIVIDAD IV - ANÁLISIS CRÍTICO SOBRE LA UTILIDAD DE FLEX	32
• Propuesta de Tokenización en Reglas de Snort	32
CONCLUSIÓN	34
BIBLIOGRAFÍA	35

INTRODUCCIÓN

El análisis léxico es la etapa inicial y esencial de cualquier proceso de compilación en el campo de la teoría de lenguajes formales y la ingeniería del software. Su objetivo principal consiste en convertir un flujo de caracteres desestructurado (código fuente) en una secuencia ordenada de elementos léxicos válidos llamados tokens, que a su vez se aíslan con valores específicos o lexemas. A pesar de que la elaboración manual de analizadores léxicos (lexers) usando expresiones regulares y autómatas finitos deterministas (AFD) brinda un control detallado, en contextos de producción académica e industrial se utilizan herramientas de metacompilación por su eficacia, facilidad para mantenerlas y menor tendencia a errores distributivos. Este informe técnico documenta la concepción formal de un lenguaje regular llamado L_{Rust} , que funciona como un subconjunto simplificado y representativo del lenguaje Rust para programación de sistemas. Además, se especifica e implementa el analizador léxico usando Flex, un metacompilador de código abierto. Este está configurado de manera nativa bajo la programación estructurada y los tipos de datos actuales en C ++, lo que asegura un desempeño elevado y una originalidad destacada durante el desarrollo.

ACTIVIDAD I - AUTÓMATAS DE PILA

Un Autómata de Pila (AP) es un modelo matemático de computación de una máquina abstracta que extiende las capacidades de los autómatas finitos tradicionales. Su evolución radica en la incorporación de una memoria auxiliar externa en forma de pila que opera bajo el principio LIFO (Last In, First Out). Esto le otorga una capacidad de almacenamiento ilimitada, aunque el flujo de control solo tiene acceso directo para lectura o modificación sobre el símbolo ubicado en el extremo superior o tope de la pila.

Formalmente, este modelo se define algebraicamente a través de una 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Donde Q representa el conjunto finito de estados internos; Σ es el alfabeto de entrada; Γ es el alfabeto de la pila; y δ es la función de transición mapeada como $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$. Los componentes finales son q_0 como estado inicial; Z_0 como el símbolo inicial y marcador de fondo de la pila; y F como el conjunto de estados de aceptación.

El formalismo establece dos metodologías funcionalmente equivalentes para determinar la validez de una cadena. El primer criterio es la aceptación por estado final, donde la cadena es válida si, al consumirse por completo, el autómata se sitúa en un estado perteneciente al conjunto F , ignorando el contenido remanente en la memoria. El segundo criterio es la aceptación por pila vacía, el cual valida la cadena si las transiciones remueven de forma exacta todos los elementos de la pila (incluyendo Z_0), haciendo irrelevante el estado final de control.

Ejemplos de Aplicación y Funcionamiento Visual

1. Reconocimiento del Lenguaje No Regular $L = \{a^n b^n \mid n \geq 1\}$

Este lenguaje clásico requiere una secuencia de símbolos "a" seguida de una cantidad idéntica de símbolos "b". Al ser el orden de magnitud infinito, es irresoluble para un autómata finito por su falta de memoria. El AP lo resuelve usando operaciones de apilamiento (push) y desapilamiento (pop).

Para la cadena válida "aabb", la traza de ejecución visual de la pila se desarrolla de la siguiente manera:

Etapas del Proceso	Flujo en la Cinta de Lectura	Contenido de la Pila	Operación Ejecutada
Inicio del proceso	a a b b	[Z0]	Inicialización (Marcador de fondo)
Lectura carácter 1	(a) a b b	[A, Z0]	Apilar (<i>Push</i> de A)
Lectura carácter 2	a (a) b b	[A, A, Z0]	Apilar (<i>Push</i> de A)
Lectura carácter 3	a a (b) b	[A, Z0]	Desapilar (<i>Pop</i> de A)
Lectura carácter 4	a a b (b)	[Z0]	Desapilar (<i>Pop</i> de A)
Fin de la cadena	a a b b (ε)	[Z0]	Aceptación por Pila Vacía / Estado Final

Por cada "a" se introduce un símbolo en memoria y, al aparecer la primera "b", el sistema cambia a un estado de comparación para hacer pop. Al concluir el flujo en épsilon (ε), la reaparición de Z0 certifica el balance y acepta la cadena.

2. Validación de Paréntesis Balanceados en Expresiones

En el diseño de compiladores, el AP es indispensable para verificar que cada símbolo de apertura posea su cierre jerárquico en expresiones aritméticas. El autómata almacena cada apertura y la elimina al encontrar su contraparte de cierre.

Para la cadena válida "(())", la memoria auxiliar evoluciona visualmente de la siguiente forma:

Etapa del Proceso	Flujo en la Cinta de Lectura	Contenido de la Pila	Operación Ejecutada
Inicio del proceso	(())	[Z0]	Inicialización (Marcador de fondo)
Lectura apertura 1	(())	[(, Z0]	Apilar (<i>Push</i> de apertura)
Lectura apertura 2	(())	[(, (, Z0]	Apilar (<i>Push</i> de apertura)
Lectura cierre 1	(())	[(, Z0]	Desapilar (<i>Pop</i> por coincidencia)
Lectura cierre 2	(()) (ε)	[Z0]	Desapilar (<i>Pop</i>) y Aceptación

El autómata ignora variables u operandos y se concentra en la estructura. Si al terminar la lectura la pila muestra el marcador base Z0, significa que la sintaxis de los bloques lógicos es correcta. A partir de estas aplicaciones, se concluye que las utilidades del autómata de pila son cruciales en la informática, ya que constituyen el núcleo de los analizadores sintácticos (parsers). Asimismo, permiten evaluar estructuras recursivas anidadas de profundidad indeterminada, como el emparejamiento de llaves en código fuente o etiquetas en formatos de datos estructurados como XML, JSON y HTML,

optimizando además la resolución de expresiones complejas basadas en notaciones polacas inversas.

Toda esta versatilidad consolida la importancia computacional del autómata de pila al analizar su posición superior dentro de la Jerarquía de Chomsky. Mientras los modelos finitos (AFD/AFND) se limitan a lenguajes regulares por su memoria estática, el AP rompe esta barrera gracias a su memoria dinámica, permitiéndole reconocer lenguajes independientes del contexto (Tipo 2). Además, a diferencia de los modelos finitos, las versiones no determinísticas del AP son estrictamente más poderosas que las determinísticas, siendo indispensables para resolver estructuras simétricas complejas sin un pivote explícito que guíe la transición del sistema.

Esta marcada diferencia justifica por completo la arquitectura modular de un compilador real, el cual divide sus tareas según la complejidad del lenguaje. El analizador léxico utiliza expresiones regulares y AFD porque solo requiere identificar patrones lineales e independientes conocidos como tokens; en contraste, la sintaxis global de un código es jerárquica y libre de contexto. En conclusión, sin la capacidad de almacenamiento estructurado del autómata de pila, los sistemas informáticos actuales serían totalmente incapaces de interpretar árboles sintácticos abstractos o asegurar la correcta precedencia algebraica en los lenguajes de programación modernos.

ACTIVIDAD II - CREACION DE LEXER PARA VERIFICACION DE ARCHIVOS DOCKERFILE MEDIANTE EXPRESIONES REGULARES

Antes de profundizar debemos conocer que Docker es una herramienta que emplea el kernel de Linux y sus funciones nativas, como los grupos de control y los espacios de nombres, para aislar procesos y ejecutarlos de forma independiente. Esto permite la creación de contenedores, los cuales son mucho más ligeros y modulares que una máquina virtual tradicional porque comparten el sistema operativo anfitrión. De esta forma, se facilita enormemente la creación, implementación, copia y distribución de aplicaciones o servicios con todas sus dependencias en múltiples entornos de forma idéntica.

Ahora, un archivo Dockerfile está estructurado por una sintaxis secuencial muy definida orientada a líneas, por lo que el lexer debe ser capaz de procesar el texto identificando las siguientes 3 tipologías de texto:

- ❖ **Instrucciones o Palabras Reservadas:** Debe ser capaz de identificar comandos clave que Docker entiende, los cuales son escritos típicamente en mayúsculas al inicio de la línea (Ejemplo: FROM, RUN, WORKDIR, COPY).
- ❖ **Argumentos y Valores:** Representan el contenido que acompaña a la instrucción, los cuales pueden ser rutas de archivos, nombres de imágenes, versiones, o comandos del sistema operativo (Ejemplo: ubuntu:20.04, /app, npm install bcrypt).
- ❖ **Comentarios y Espacios:** Líneas que inician con # y espacios en blanco o tabulaciones, los cuales el lexer debe reconocer para ignorarlos durante la validación estricta de sintaxis.

Una vez conocidas las tipologías de texto que nuestro lexer debe conocer, podemos diseñar el diccionario de tokens y expresiones regulares que se encargaran de dar validez algorítmica, por lo que a continuación se muestra la definición de dicho diccionario:

DICCIONARIO DE TOKENS Y EXPRESIONES REGULARES PARA DOCKERFILES		
Nombre del Token	Expresión Regular (Regex)	Descripción / Lexema Esperado
TK_COMMENT	#.*	Identifica comentarios para ser ignorados por el lexer.
TK_SPACE	[\t]+	Identifica espacios en blanco y tabulaciones.
TK_NEWLINE	\n	Detecta los saltos de línea.
TK_FROM	\bFROM\b	Instrucción base para definir la imagen.
TK_RUN	\bRUN\b	Instrucción para ejecutar comandos de terminal.
TK_WORKDIR	\bWORKDIR\b	Instrucción para establecer el directorio de trabajo.
TK_COPY	\bCOPY\b	Instrucción para copiar archivos al contenedor.
TK_CMD	\bCMD\b	Instrucción que define el comando por defecto al iniciar.
TK_STRING	"[^"]*"	Captura cadenas de texto encerradas entre comillas dobles.
TK_ARGUMENT	[a-zA-Z0-9_./:\-]+	Captura argumentos generales (rutas, versiones, nombres).
TK_MISMATCH	.	Atrapa cualquier carácter no reconocido para detonar un error léxico.

Habiendo definido el diccionario de tokens y expresiones regulares podemos realizar la implementación en el lenguaje de programación Python, donde haremos uso de la librería nativa de expresiones regulares **re**, con la cual se evita el uso de dependencias externas al solo involucrar al intérprete básico de Python.

A continuación se presenta el código fuente de la implementación del analizador léxico el cual es capaz de leer el texto, extraer los lexemas y manejar los errores léxicos según las expresiones regulares definidas en el diccionario:

```
import re
import sys

# Definicion del diccionario de tokens y expresiones regulares para Dockerfile
# a traves de una tupla

tokens_docker = [
    # ELEMENTOS IGNORABLES Y DE CONTROL
    ('TK_COMMENT', r'#. *'),
    ('TK_SPACE', r'[\t ]+'),
    ('TK_NEWLINE', r'\n'),

    # PALABRAS RESERVADAS (Específicas)
    ('TK_FROM', r'\bFROM\b'),
    ('TK_RUN', r'\bRUN\b'),
    ('TK_WORKDIR', r'\bWORKDIR\b'),
    ('TK_COPY', r'\bCOPY\b'),
    ('TK_CMD', r'\bCMD\b'),

    # ESTRUCTURAS CERRADAS
    ('TK_STRING', r'"[^"]*"'),

    # VALORES GENERICOS
    ('TK_ARGUMENT', r'[a-zA-Z0-9_./:\-]+'),

    # MANEJO DE ERRORES
    ('TK_MISMATCH', r'.')
]
```

```

# Funcion principal del Analizador Lexico
def lexer(texto_fuente):

    # Unir todas las regex en una sola utilizando grupos nombrados de Python
    regex_maestra = '|'.join(f'(?P<{nombre}>{patron})' for nombre, patron in
tokens_docker)

    linea_actual = 1
    inicio_linea = 0

    print("*** INICIO DEL ANALISIS LEXICO ***")

    # Se emplea la funcion finditer que escanea el string de izquierda a derecha buscando
coincidencias
    for coincidencia in re.finditer(regex_maestra, texto_fuente):

        # Se obtiene el tipo de token y el lexema correspondiente a la coincidencia
encontrada
        tipo_token = coincidencia.lastgroup
        lexema = coincidencia.group(tipo_token)

        # Si se encuentra un salto de linea, se actualiza la linea actual y el inicio de
la línea
        if tipo_token == 'TK_NEWLINE':
            inicio_linea = coincidencia.end()
            linea_actual += 1

        elif tipo_token in ['TK_SPACE', 'TK_COMMENT']:
            continue # Ignoramos los espacios y los comentarios de Docker

        elif tipo_token == 'TK_MISMATCH':

            # Si se encuentra un token no reconocido,
            # se indica el error lexico y se detiene la ejecucion del programa

            columna = coincidencia.start() - inicio_linea
            print(f"\n[!] ERROR LEXICO: Caracter no reconocido '{lexema}' en la linea
{linea_actual}, columna {columna}")
            sys.exit(1)

        else:
            # Token valido encontrado
            columna = coincidencia.start() - inicio_linea
            print(f"Token: {tipo_token:<15} | Lexema: {lexema:<20} | Línea: {linea_actual}")

```

```

#Lectura del archivo dockerfile
def cargar_archivo(nombre_archivo):

    try:
        with open(nombre_archivo, 'r', encoding='utf-8') as archivo:
            return archivo.read()

    except FileNotFoundError:
        print(f"Error: El archivo '{nombre_archivo}' no fue encontrado.")
        sys.exit(1)

# Punto de entrada del script
if __name__ == '__main__':

    # Validamos que se pase el archivo dockerfile por argumento
    if len(sys.argv) < 2:

        print("Uso correcto: python lexer_docker.py <archivo_dockerfile>")
        sys.exit(1)

    ruta_archivo = sys.argv[1]
    codigo_fuente = cargar_archivo(ruta_archivo)

    lexer(codigo_fuente)
    print("*** FIN DEL ANALISIS LEXICO ***")

```

CONSTRUCCIÓN DEL CÓDIGO

Importación de Módulos Nativos del Sistema

```

import re
import sys

```

RE, proporciona el motor para el procesamiento y emparejamiento de cadenas mediante patrones complejos

SYS, permite interactuar directamente con el entorno operativo del sistema. Se utiliza para capturar los argumentos pasados por la CLI y abortar la ejecución del programa cuando ocurre una excepción léxica.

Configuración de la Jerarquía Léxica

```
tokens_docker = [  
    # ELEMENTOS IGNORABLES Y DE CONTROL  
    ('TK_COMMENT', r'#.*'),  
    ('TK_SPACE', r'[\t ]+'),  
    ('TK_NEWLINE', r'\n'),  
  
    # PALABRAS RESERVADAS (Específicas)  
    ('TK_FROM', r'\bFROM\b'),  
    ('TK_RUN', r'\bRUN\b'),  
    ('TK_WORKDIR', r'\bWORKDIR\b'),  
    ('TK_COPY', r'\bCOPY\b'),  
    ('TK_CMD', r'\bCMD\b'),  
  
    # ESTRUCTURAS CERRADAS  
    ('TK_STRING', r'"[^"]*"),  
  
    # VALORES GENERICOS  
    ('TK_ARGUMENT', r'[a-zA-Z0-9_./:\- ]+'),  
  
    # MANEJO DE ERRORES  
    ('TK_MISMATCH', r'.')  
]
```

La lista **tokens_docker**, almacena tuplas compuestas por el Token y su expresión regular.

Los elementos siguen un orden jerárquico crítico para garantizar la validez del análisis compuesto de la siguiente manera:

Prioridad de Descarte Temprano: Al situar `TK_COMMENT`, `TK_SPACE` y `TK_NEWLINE` al inicio de la lista, el motor evalúa y filtra inmediatamente los caracteres más irrelevantes para la sintaxis.

Palabras Reservadas Específicas Las directivas de Docker como `FROM`, `RUN`, etc. se declaran utilizando delimitadores de frontera de palabra `\b`, lo que impide que cadenas mayores que contengan estas subcadenas causen falsos positivos.

Estructuras Cerradas e Identificadores Genéricos: Se sitúan debajo de las palabras reservadas, ya que si una palabra clave como `FROM` se evaluara después de `TK_ARGUMENT`, el patrón genérico `[a-zA-Z0-9_./:\-]+` la absorbería erróneamente como un argumento común.

Manejador de Errores Final: El token `TK_MISMATCH` utiliza el punto (`.`), que captura cualquier carácter individual. Al ubicarse al final, funciona como un estado de absorción fallido: si el carácter evaluado no coincidió con ninguna regla superior, cae obligatoriamente en esta categoría, detonando el error léxico.

EXPRESION REGULAR MAESTRA

```
regex_maestra = '|'.join(f'(?P<{nombre}>{patron})' for nombre, patron in tokens_docker)
```

Esta línea compila todas las reglas individuales en una sola gran expresión regular. Utiliza la sintaxis de grupos nombrados de Python (`?P<Nombre>patrón`). La función `'|'.join(...)` concatena secuencialmente cada grupo con el operador lógico alternativo **OR** para obtener como resultado un string unificado que permite evaluar el texto fuente en una

sola pasada lineal, mapeando automáticamente los lexemas detectados con sus respectivos nombres de token.

Variables de Control

```
línea_actual = 1
inicio_línea = 0
```

Se definen dos variables de control posicional donde **línea_actual** actúa como un contador de líneas absoluto. **inicio_línea** guarda el índice del carácter donde da comienzo la línea que se está procesando actualmente; este puntero es indispensable para calcular la columna exacta de cada lexema restando este origen a la posición absoluta de la coincidencia.

Bucle de Tokenización Secuencial

```
for coincidencia in re.finditer(regex_maestra, texto_fuente):
```

Aquí se hace uso de la función **re.finditer()**, para escanear la cadena de texto de izquierda a derecha devolviendo objetos de coincidencia a lo largo del ciclo.

Extracción de Lexemas y Tokenización

```
tipo_token = coincidencia.lastgroup
lexema = coincidencia.group(tipo_token)
```

La función **coincidencia.lastgroup**, recupera el nombre del grupo regex específico que logró el emparejamiento con el texto evaluado (por ejemplo, devuelve la cadena 'TK_FROM').

La función `coincidencia.group(tipo_token)`, extrae directamente del archivo fuente la subcadena exacta de texto (o lexema) que activó dicho grupo.

FLUJO DE CONTROL Y FALLOS

Manejo de Estructura de línea:

```
if tipo_token == 'TK_NEWLINE':  
    inicio_linea = coincidencia.end()  
    linea_actual += 1
```

Cuando se intercepta un salto de línea (`\n`), el puntero `inicio_linea` se desplaza inmediatamente al índice posterior de dicho carácter `coincidencia.end()` y se incrementa el contador de líneas.

Filtrado de Elementos No Operacionales

```
elif tipo_token in ['TK_SPACE', 'TK_COMMENT']:  
    continue # Ignoramos los espacios y los comentarios de Docker
```

Al detectar espacios en blanco o comentarios, la sentencia `continue` aborta el ciclo actual y avanza al siguiente componente léxico sin imprimir datos en la consola, logrando el descarte de elementos irrelevantes para la sintaxis.

Manejo de Error Léxico

```
elif tipo_token == 'TK_MISMATCH':  
  
    # Si se encuentra un token no reconocido, se indica el error léxico y se detiene la ejecución del  
    programa  
    columna = coincidencia.start() - inicio_linea
```



```
print(f"\n ERROR LEXICO: Caracter no reconocido '{lexema}' en la línea {linea_actual}, columna {columna}")
sys.exit(1)
```

Si el flujo alcanza esta regla, determina que el carácter es inválido y calcula la columna restando la posición de inicio menos la posición de la línea actual, muestra el mensaje e interrumpe la ejecución mediante `sys.exit(1)`.

Registro de componentes Validos

```
else:
    # Token valido encontrado
    columna = coincidencia.start() - inicio_linea
    print(f"Token: {tipo_token:<15} | Lexema: {lexema:<20} | Línea: {linea_actual}")
```

Cualquier elemento que no pertenezca a los casos anteriores, se considera como un token valido. Se calcula su columna y se genera el formato de la salida donde se expone el token, el lexema detectado y su ubicación exacta en el código fuente.

Carga de Archivos

```
# Lectura del archivo dockerfile
def cargar_archivo(nombre_archivo):

    try:
        with open(nombre_archivo, 'r', encoding='utf-8') as archivo:
            return archivo.read()

    except FileNotFoundError:
        print(f"Error: El archivo '{nombre_archivo}' no fue encontrado.")
        sys.exit(1)
```

La función permite la lectura física del archivo encapsulándola en un bloque seguro **try-except**. La declaración **with** garantiza la liberación automática de los descriptores de archivo en el sistema operativo una vez concluida la lectura o si ocurre un fallo inesperado.

El parámetro `encoding='utf-8'` mitiga errores de codificación ante la presencia de caracteres especiales o acento.

Validador del punto de entrada del CLI

```
if __name__ == '__main__':  
  
    # Validamos que se pase el archivo dockerfile por argumento  
    if len(sys.argv) < 2:  
  
        print("Uso correcto: python lexer_docker.py <archivo_dockerfile>")  
        sys.exit(1)  
  
    ruta_archivo = sys.argv[1]  
    codigo_fuente = cargar_archivo(ruta_archivo)  
  
    lexer(codigo_fuente)  
    print("\n--[ FIN DEL ANALISIS LEXICO ]--")
```

En este punto el bloque `__name__ == '__main__'` restringe la ejecución automática del código si este es importado como módulo en un sistema superior. Actúa como la interfaz de línea de comandos inicial: verifica que el usuario haya provisto el parámetro obligatorio (la ruta del archivo a validar), captura dicho argumento desde el vector `sys.argv[1]`, invoca la carga segura de datos y transfiere la cadena completa resultante al motor del analizador léxico para su resolución final.

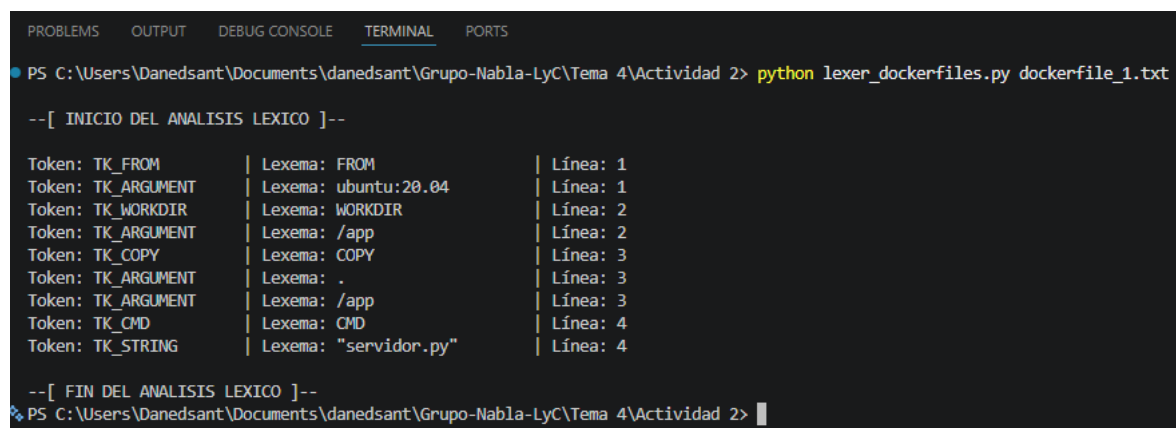
EJEMPLOS DE EJECUCIÓN

Ejemplo 1 – Dockerfile_1.txt - Sintaxis Básico Correcta

```
FROM ubuntu:20.04
WORKDIR /app
COPY . /app
CMD "servidor.py"
```

En este ejemplo se tiene una sintaxis básica correcta donde se espera obtener por salida en consola todos los lexemas y tokens detectados por los que está compuesto este dockerfile.

Ejemplo de Inicialización y Ejecución del dockerfile_1.txt



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> python lexer_dockerfiles.py dockerfile_1.txt

--[ INICIO DEL ANALISIS LEXICO ]--

Token: TK_FROM      | Lexema: FROM      | Línea: 1
Token: TK_ARGUMENT  | Lexema: ubuntu:20.04 | Línea: 1
Token: TK_WORKDIR   | Lexema: WORKDIR   | Línea: 2
Token: TK_ARGUMENT  | Lexema: /app      | Línea: 2
Token: TK_COPY      | Lexema: COPY      | Línea: 3
Token: TK_ARGUMENT  | Lexema: .         | Línea: 3
Token: TK_ARGUMENT  | Lexema: /app      | Línea: 3
Token: TK_CMD       | Lexema: CMD       | Línea: 4
Token: TK_STRING    | Lexema: "servidor.py" | Línea: 4

--[ FIN DEL ANALISIS LEXICO ]--
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> |
```

Ejemplo 2 – dockerfile_2.txt – Filtrado de Comentarios

```
# Instalacion de Node
FROM node:18-alpine

RUN npm install
```

Para este ejemplo se tiene un código que incluye el uso de comentarios, al cual aplicaremos la ejecución del analizador léxico el cual filtra estos comentarios de manera exitosa y muestra por salida los lexemas relevantes de la sintaxis.

Ejemplo de Inicialización y Ejecución del dockerfile_2.txt

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> python lexer_dockerfiles.py dockerfile_2.txt

--[ INICIO DEL ANALISIS LEXICO ]--

Token: TK_FROM      | Lexema: FROM      | Línea: 2
Token: TK_ARGUMENT  | Lexema: node:18-alpine | Línea: 2
Token: TK_RUN       | Lexema: RUN       | Línea: 4
Token: TK_ARGUMENT  | Lexema: npm       | Línea: 4
Token: TK_ARGUMENT  | Lexema: install   | Línea: 4

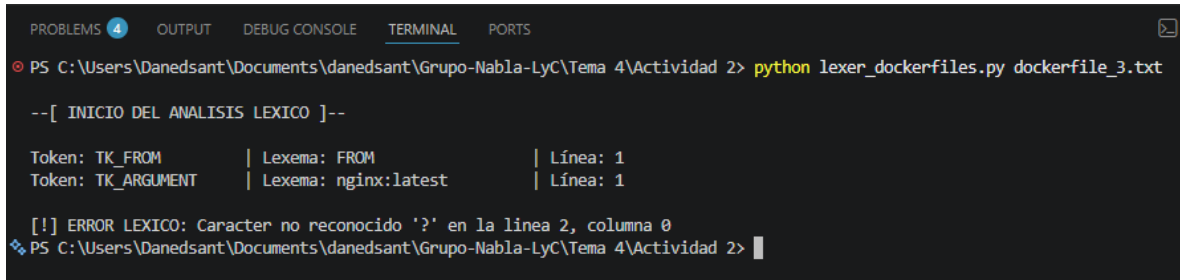
--[ FIN DEL ANALISIS LEXICO ]--
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> █
```

Ejemplo 3 – dockerfile_3.txt – Manejo de Errores

```
Tema 4 > Actividad 2 > dockerfile_3.txt
1 FROM nginx:latest
2 ?COPY index.html /usr/share/nginx/html/
```

Para este ejemplo tenemos el caso donde incluimos junto a una directiva de docker **COPY** un signo de interrogación, ? El cual no está incluido en las reglas del diccionario, por lo que se espera que durante la ejecución, se detecte esto como un error léxico y sea capturado mostrando su ubicación en el código de manera exitosa deteniendo la ejecución en ese momento.

Ejemplo de Inicialización y Ejecución del dockerfile_3.txt



```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> python lexer_dockerfiles.py dockerfile_3.txt

--[ INICIO DEL ANALISIS LEXICO ]--

Token: TK_FROM      | Lexema: FROM      | Línea: 1
Token: TK_ARGUMENT  | Lexema: nginx:latest | Línea: 1

[!] ERROR LEXICO: Caracter no reconocido '?' en la línea 2, columna 0
PS C:\Users\Danedsant\Documents\danedsant\Grupo-Nabla-LyC\Tema 4\Actividad 2> |
```

Como se puede observar en las capturas de la ejecución, el analizador léxico logra capturar el error y muestra por pantalla la ubicación del mismo deteniendo la ejecución en el momento.

Actividad III - Descripción Formal del Lenguaje Diseñado (L_{Rust})

El lenguaje L_{Rust} es un lenguaje regular libre de contexto diseñado específicamente para capturar las estructuras de control, tipos de datos primarios y sintaxis de asignación características de Rust. El alfabeto de entrada Σ está compuesto por caracteres ASCII alfanuméricos y símbolos matemáticos/puntuación estándar.

A continuación, se presenta la especificación detallada de los componentes léxicos asociados al lenguaje:

Palabras Reservadas (Keywords): Definidas bajo patrones fijos de precedencia léxica absoluta.

fn: Define el punto de entrada o declaración de funciones.

let: Palabra clave obligatoria para ligaduras de variables.

mut: Especificador explícito de mutabilidad.

if, else: Estructuras de bifurcación condicional.

return: Declaración de retorno de flujo.

true, false: Literales booleanos nativos.

Tipos de Datos Primitivos:

i32: Entero con signo de 32 bits.

f32: Flotante de precisión simple de 32 bits.

bool: Tipo lógico booleano.

Operadores (Aritméticos y Relacionales):

Asignación: =

Aritméticos: +, -, *, /

Relacionales: ==, !=, <, >

Signos de Puntuación y Delimitadores:

Límites de bloque: {, }

Agrupamiento de expresiones/argumentos: (,)

Declaración de tipo / Coerción: :

Retorno de tipo de función: →

Terminación de sentencia: ;

Separador de elementos: ,

Identificadores: Cadenas de caracteres que responden a la expresión regular: [a-zA-Z_][a-zA-Z0-9_]*.

Literales Numéricos:

Entero: Secuencia de dígitos [0-9]+.

Flotante: Punto flotante estricto [0-9]+\.[0-9]+.

Elementos Descartables (No-Tokens):

Comentarios de una línea: Identificados por el patrón `//.*` y descartados del flujo de tokens.

Espacios en blanco y tabulaciones: `[\t\r\n]+`, utilizados únicamente para calcular la posición exacta de línea y columna del buffer de lectura.

Manual de Usuario del Metacompilador Flex

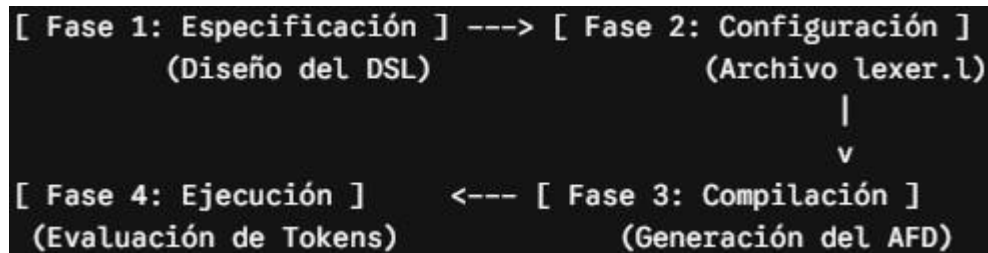
Flex (Fast Lexical Analyzer Generator) es una herramienta matemática y de ingeniería de software encargada de generar escáneres o analizadores léxicos a partir de especificaciones basadas en expresiones regulares. El archivo de entrada clásica posee la extensión `.l` y estructuralmente se divide en tres secciones claramente delimitadas por el delimitador `%%`:

1. **Sección de Definiciones (Declaraciones):** Ubicada al inicio del archivo. Permite la inclusión de código directo en el lenguaje destino (en este caso C++) delimitado por `%{` y `%}`. Aquí se importan librerías estándar (`iostream`, `string`), se definen macros de expresiones regulares comunes y variables globales de control (como el conteo de líneas y columnas).
2. **Sección de Reglas de Transición:** Es el núcleo del archivo. Consiste en una tabla bidimensional implícita donde la columna izquierda define un patrón mediante una Expresión Regular y la columna derecha encierra entre llaves `{ }` la acción semántica (código C++) que se ejecutará inmediatamente cuando el motor de emparejamiento de Flex reconozca dicho patrón. Flex utiliza el algoritmo de coincidencia más larga (Maximal Munch); en caso de empate, ejecuta la regla que aparezca primero en el archivo.

3. **Sección de Código de Usuario:** Contiene rutinas de soporte escritas directamente en C++. En esta sección se codifica la función principal `main()`, la cual invoca de manera iterativa a la subrutina `yylex()` generada automáticamente por Flex. `yylex()` lee el archivo de entrada (`yyin`) carácter a carácter y retorna los códigos simbólicos de cada token detectado.

Metodología y Proceso de Creación del Analizador Léxico para L_{Rust}

La construcción del analizador léxico automatizado para el subconjunto de lenguajes L_{Rust} no se realizó mediante codificación manual carácter por carácter, sino que siguió un proceso de ingeniería de software estructurado basado en la traducción de especificaciones formales. El flujo metodológico se divide en cuatro fases fundamentales:



Fase 1: Especificación y Diseño del Alfabeto L_{Rust}

El proceso inició con la delimitación formal del Lenguaje de Dominio Específico (DSL). Se extrajeron las palabras reservadas (`fn`, `let`, `mut`, `if`, `else`, `return`), los operadores (`-`, `>`, `==`, `=`, `+`) y los delimitadores propios de la gramática de Rust. Cada componente se asoció unívocamente con un identificador de token (ej. `TK_FN`, `TK_ARROW`), sentando las bases del diccionario léxico.

Fase 2: Configuración del Archivo de Entrada de Flex (`lexer.l`)

Una vez definido el alfabeto, se procedió a transcribir el comportamiento del lexer en la sintaxis nativa de Flex dentro del archivo `lexer.l`, estructurando en sus secciones reglamentarias:

1. **Zona de Declaraciones (%{ ... %}):** Se incluyeron las cabeceras de C++ (`<iostream>`, `<string>`) y se inicializan las variables globales encargadas del control espacial del buffer (`line_num` y `col_num`).
2. **Definición de Reglas y Acciones:** Se acopló cada expresión regular con un bloque de código C++. En esta fase fue crítico el orden de aparición: las palabras clave se posicionaron por encima de los identificadores genéricos para asegurar que el algoritmo de coincidencia (Maximal Munch) prioriza los tokens reservados del sistema frente a variables creadas por el usuario.

Fase 3: Generación del Autómata y Compilación Nativa

Con el archivo de especificación listo, el proceso de transformación automatizada se ejecuta en dos pasos de traducción:

1. **Traducción de Flex a C++:** Al ejecutar el comando `flex -o lex.yy.cc lexer.l`, el motor matemático de Flex procesa las expresiones regulares y las compila internamente en un Autómata Finito Determinista (AFD). Este AFD se traduce en matrices y tablas de transición indexadas estáticamente dentro del código fuente intermedio `lex.yy.cc`.
2. **Compilación del Binario:** El archivo `lex.yy.cc` es compilado por el compilador nativo `g++`. Este paso empaqueta las rutinas de bajo nivel del buffer con la función principal `main()`, generando el ejecutable final optimizado para el sistema operativo.

Fase 4: Flujo de Ejecución y Consumo del Buffer

Al ponerse en marcha el analizador pasándole el archivo programa_prueba.rs, el sistema opera de la siguiente manera:

- La subrutina `yylex()` toma el control y abre el flujo de lectura (`yyin`).
- El texto se procesa carácter por carácter a través de las tablas de transición del AFD.
- Cada vez que se reconoce un patrón válido, el lexema se aísla en la variable interna `yytext`, se ejecuta la función de localización `update_location()` para desplazar las coordenadas de la matriz, y se retorna el token correspondiente a la consola.
- Si se procesan elementos descartables (como comentarios `//.*` o espacios), el lexer actualiza las posiciones pero no retorna ningún token, limpiando eficazmente el flujo para la futura etapa de análisis sintáctico.

Contenido del Archivo README.md (Instalación e Implementación)

A continuación se transcribe el documento de ingeniería de despliegue que debe acompañar al repositorio en formato Markdown:

Analizador Léxico para `LRust` mediante Flex (C++)

Este módulo contiene la especificación léxica e implementación automatizada de un lexer para el subconjunto de lenguaje `LRust`, desarrollado en C++ utilizando la herramienta de metacompilación Flex.

Prerrequisitos del Sistema

Para compilar y ejecutar este software de manera local, es mandatorio contar con las siguientes herramientas instaladas en su entorno de ejecución (Linux / WSL recomendado):

1. **Flex:** Generador de analizadores léxicos.
2. **GCC / G++:** Compilador nativo de C++.

Instalación de Dependencias (Ubuntu)

```
```bash
```

```
sudo apt update
```

```
sudo apt install flex build-essential -y
```

### **Instalación de Dependencias (Windows)**

Para compilar y ejecutar este proyecto en Windows, se requiere configurar manualmente las herramientas de compilación o utilizar una terminal emulada. Siga uno de los siguientes métodos:

#### **Método 1:** Configuración Manual (Recomendado para tu entorno actual)

- **Flex:** Descargue e instale el paquete binario de Flex para Windows desde el proyecto GnuWin32 (instalado típicamente en `C:\Program Files (x86)\GnuWin32\bin\flex.exe`).
- **Compilador C++ (MinGW/GCC):** Asegúrese de contar con un entorno GCC para Windows, como el incluido en **Dev-C++** o **MinGW-w64** (ubicado típicamente en `C:\Dev-Cpp\MinGW64\bin\g++.exe`).

- **Variable de Entorno (Opcional):** Se recomienda añadir ambas rutas binarias al ``PATH`` del sistema para poder invocar directamente los comandos ``flex`` y ``g++`` sin necesidad de escribir sus rutas absolutas.

## **Método 2:** Usando el Gestor de Paquetes Chocolatey (Alternativa Automatizada)

Si tiene instalado el gestor de paquetes Chocolatey en la PowerShell de Windows, puede instalar ambas herramientas ejecutando:

```
``bash
```

```
choco install winflexbison mingw -y
```

## **Estructura de Archivos**

**lexer.l:** Archivo de especificación de Flex (Contiene definiciones, expresiones regulares y acciones en C++).

**programa\_prueba.rs:** Archivo de prueba con código escrito bajo las reglas sintácticas de `LRust`.

## **Instrucciones de Compilación y Ejecución**

Siga rigurosamente el orden de los siguientes comandos en su terminal:

### **1. Generar el código fuente de C++ desde la especificación de Flex:**

Este comando compila las expresiones regulares en un autómata y exporta el analizador como un archivo fuente de C++ reproducible.

```
flex -o lex.yy.cc lexer.l
```

## **2. Compilar el código fuente generado junto al motor de ejecución:**

Se utiliza el compilador de C++ para empaquetar el binario optimizado.

```
g++ lex.yy.cc -o lexer_rust
```

## **3. Ejecutar el analizador léxico pasando un archivo de código fuente:**

```
./lexer_rust programa_prueba.rs
```

## **Gestión de Errores Léxicos**

El analizador léxico está programado con un estado de fallo dinámico (TK\_ERROR). Si encuentra un carácter no definido en el alfabeto de  $L_{Rust}$ , el programa no colapsará; en su lugar, imprimirá un reporte en consola indicando el carácter inválido y la ubicación exacta (línea y columna), reanudando el análisis de inmediato desde el estado inicial del autómata.

## **Selección del Lenguaje de Dominio Específico (DSL)**

Para el estudio de la aplicación de herramientas de metacompilación en el área de la seguridad informática, se ha seleccionado el lenguaje de Reglas de Snort / Suricata. Este lenguaje es un DSL estructurado de manera formal, utilizado globalmente por Sistemas de Detección y Prevención de Intrusos (IDS/IPS) para realizar una inspección profunda de paquetes (DPI) en redes de datos.

Una regla de Snort se compone estrictamente de dos bloques sintácticos bien diferenciados:

- La Cabecera de la Regla (Rule Header): Define la acción a tomar, el protocolo de red (tcp, udp, icmp, etc.), las direcciones IP de origen/destino y los puertos asociados.
- Las Opciones de la Regla (Rule Options): Se encuentran delimitadas entre paréntesis y contienen los metadatos de la regla, las directivas de inspección de contenido (content) y los identificadores únicos (sid).

## Actividad IV - Análisis Crítico y Reflexión sobre la Utilidad de FLEX

El análisis de tráfico en redes de alta velocidad (infraestructuras de ordenadores que operan a velocidades mayores a 1 Gbps) exige que el motor de detección procese flujos masivos de caracteres de manera continua y en tiempo real. La implementación manual de un analizador léxico o el uso de bibliotecas de expresiones regulares basadas en software interpretado (como los módulos tradicionales de Python) generan cuellos de botella críticos debido a la sobrecarga computacional que implica el retroceso (backtracking).

La utilización del metacompilador FLEX mitiga este problema de forma óptima. A partir de una especificación léxica formal, FLEX genera un programa en lenguaje C nativo optimizado a bajo nivel, estructurado bajo un Autómata Finito Determinista (AFD). Las ventajas fundamentales de este enfoque en el ecosistema de la ciberseguridad son:

**Complejidad Temporal Lineal  $O(n)$ ,** donde el tiempo requerido para evaluar una cadena de caracteres depende exclusivamente de la longitud del texto de entrada ( $n$ ), garantizando un rendimiento constante sin importar la cantidad de reglas cargadas en el sistema.

**Eficiencia de Memoria:** Al compilar los patrones léxicos en tablas de transición indexadas estáticamente, se suprime el consumo dinámico de memoria durante la fase de análisis, condición indispensable para sistemas embebidos de red y cortafuegos.

### Propuesta de Tokenización

Para validar el diseño léxico propuesto, se toma como entrada una regla real de Snort orientada a la detección de tráfico malicioso en el servicio SSH:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"Intrusión SSH"; sid:100001;)
```



El analizador léxico generado mediante la herramienta FLEX descompone la cadena de entrada carácter a carácter, produciendo la siguiente secuencia de componentes léxicos (Tokens y Lexemas):

Token	Lexema	Expresión Regular Regular (Regex)	Descripción del Componente
TK_ACCION	alert	alert log drop pass	Determina la acción administrativa ante la coincidencia.
TK_PROTOCOLO	tcp	tcp udp icmp ip	Especifica el protocolo de la capa de transporte o red.
TK_VARIABLE	\$EXTERNAL_NET	\\$[a-zA-Z][a-zA-Z0-9_]*	Identificador de variable interna (inicia con \$).
TK_ANY	any	any	Comodín que representa cualquier puerto o dirección IP.
TK_DIRECCION	->	->	Operador que define la dirección del flujo de red.
TK_VARIABLE	\$HOME_NET	\\$[a-zA-Z][a-zA-Z0-9_]*	Identificador de la variable de la red local.
TK_PUERTO	22	[0-9]+	Valor numérico entero que representa el puerto de destino.
TK_LPAREN	(	\(	Carácter delimitador de apertura para la sección de opciones.
TK_OPC_KEY	msg:	[a-zA-Z][a-zA-Z0-9_]*:	Palabra clave que define una opción de inspección.
TK_STRING	"Intrusión SSH"	\"[^"]*"	Cadena de texto literal encerrada entre comillas dobles.
TK_PUNTO_Y_COMA	;	;	Delimitador de fin de una opción particular.
TK_OPC_KEY	sid:	[a-zA-Z][a-zA-Z0-9_]*:	Clave para el identificador único de la firma (Snort ID).
TK_NUMERO	100001	[0-9]+	Identificador numérico secuencial de la regla.
TK_RPAREN	)	\)	Carácter delimitador de cierre de la sección de opciones.

## CONCLUSIÓN

A lo largo de este informe se ha podido comprobar empírica y teóricamente el papel fundamental del análisis léxico en el procesamiento de lenguajes formales. A través de las fases prácticas, se demostró que la elección de la herramienta de tokenización debe alinearse con la complejidad estructural del lenguaje destino.

En primer lugar, el estudio de los Autómatas de Pila (AP) evidenció que, aunque los Autómatas Finitos (AFD) son eficientes para la tokenización lineal, la memoria dinámica del AP es indispensable para el análisis sintáctico de estructuras anidadas e independientes del contexto. Por otro lado, la construcción nativa del analizador para *Dockerfile* comprobó que lenguajes regulares con sintaxis secuenciales planas se procesan con alta eficiencia utilizando librerías nativas como las de Python, permitiendo un manejo robusto de errores sin dependencias externas.

Sin embargo, al escalar hacia lenguajes más complejos como el subconjunto *LRust*, la implementación manual se vuelve propensa a errores. En este nivel, el uso de metacompiladores como Flex es el estándar industrial, generando autómatas en C++ optimizados a bajo nivel. Finalmente, la extrapolación de estas herramientas a la ciberseguridad, específicamente en la evaluación de reglas de Snort, resalta su valor crítico: mantener una complejidad temporal lineal garantiza una inspección profunda de paquetes en tiempo real, consolidando al análisis léxico como un pilar de la ciberdefensa moderna.

## BIBLIOGRAFÍA

**Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008).** *Compiladores: Principios, técnicas y herramientas* (2da ed.). Ciudad de México, México: Pearson Educación.

**Levine, J. (2009).** *Flex & Bison: Text Processing Tools* [Herramientas de procesamiento de texto] (1ra ed.). Sebastopol, CA: O'Reilly Media.

**Python Software Foundation. (2026).** *re — Regular expression operations*. Python 3.14 Documentation. <https://docs.python.org/3/library/re.html>

**Docker Inc. (2026).** *Dockerfile reference: Instructions and syntax grammar*. Docker Documentation. <https://docs.docker.com/>